

Drone-Guided Autonomous Navigation: A Cooperative UAV–AMR System for GPS-Denied Environments

David Ortiz, Alfredo Romo, Noé Rosales, Sebastián Pulido, Jorge González

School of Engineering and Sciences

Tecnológico de Monterrey

Guadalajara, Mexico

[TODO: emails / matrículas]

Abstract—Many autonomous ground robots must operate where satellite positioning is unavailable or unreliable: indoor warehouses and underground facilities receive no signal, while container yards, dense urban canyons, and forest or disaster sites suffer multipath, occlusion, and blockage that degrade GPS at ground level. We present a cooperative unmanned-aerial-vehicle (UAV) and autonomous-mobile-robot (AMR) system that solves goal-directed navigation in such a GPS-denied setting. The two agents are complementary: the UAV has access to absolute positioning at altitude (provided in our testbed by a motion-capture system acting as a GPS surrogate) but cannot transport cargo, while the AMR carries no absolute-positioning instrumentation by design and therefore cannot independently determine the global location of its goal. Our pipeline has the UAV survey an unmapped 4×4 m arena from above, build a metric occupancy map through a similarity-transform image-stitching pipeline, and localize ArUco goal markers in the world frame; this aerial prior is then handed to the AMR, which fuses it with live LiDAR sensing to plan and execute a collision-free path. Core technical contributions include a from-scratch extended Kalman filter with an adaptive visual-odometry outlier gate, a priority-weighted map-fusion rule, an independent automatic-emergency-braking layer, and an application-layer cryptographic middleware securing all inter-robot traffic. The system is implemented entirely in ROS 2 across three networked compute platforms. [TODO: Headline quantitative results.]

Index Terms—GPS-denied navigation, cooperative robotics, UAV–AMR, ROS 2, extended Kalman filter, image stitching, occupancy mapping, ArUco localization, autonomous emergency braking, robot cybersecurity.

I. INTRODUCTION

A. Motivation and Problem Context ✓

Modern logistics increasingly relies on autonomous ground vehicles for material transport. In structured indoor warehouses this is well understood, but a large class of industrially relevant environments—maritime ports, container yards, and partially enclosed industrial sites—combine the openness of outdoor operation with the GPS degradation of dense infrastructure. At ground level, satellite positioning in these settings suffers from multipath reflections off metal containers and cranes, partial sky occlusion, and signal blockage, making it unreliable as the sole basis for navigation. A robot operating

there cannot assume it knows where it is, nor where its destination lies, in any global frame.

This project targets exactly that regime: autonomous navigation toward a goal whose global position is not known a priori to the ground vehicle, in an environment for which no map exists in advance.

B. Problem Statement ✓

We consider a 4×4 m arena populated with static obstacles and several ArUco fiducial markers that denote goal/zone locations and visual landmarks. An AMR must navigate from a start pose to a marked goal without colliding with obstacles. Two constraints define the difficulty. First, no map of the arena is available before the mission—it must be constructed at runtime. Second, the AMR carries *no* absolute-positioning instrumentation: it has wheel encoders, an inertial measurement unit (IMU), a 2D LiDAR, and a camera, but nothing that fixes its pose in a global frame. This is a deliberate modelling choice that mirrors the GPS-denied ground robot of the motivating scenario. Consequently the AMR cannot, on its own, determine the global position of a visual landmark marker it has not yet seen.

C. Proposed Approach

We resolve the deadlock through cooperation with a second agent that *does* have absolute positioning, but at altitude. A UAV equipped with a downward camera and tracked by a motion-capture system (acting as a surrogate for the absolute positioning a real outdoor drone would obtain from GNSS) surveys the arena from above. From its georeferenced aerial video it (i) builds a metric occupancy grid of the arena and (ii) localizes the ArUco goal markers in the world frame. This aerial prior—map plus goal pose—is handed to the AMR, which fuses it with its own live LiDAR occupancy map, plans a path with A^* , and tracks that path with a feedback-linearizing controller. Throughout, the AMR maintains a local pose estimate from a custom extended Kalman filter (EKF) that fuses wheel, IMU, and visual-odometry velocity, re-anchored when goal markers are observed.

D. Contributions

The system integrates a number of subsystems, each documented in depth in this report:

- A UAV position controller and scan-trajectory generator that tracks a boustrophedon survey pattern under motion-capture feedback, with a sliding-window fault detector robust to intermittent tracking dropouts (Sec. VI).
- An aerial image-stitching pipeline that builds a metric occupancy map using a similarity-transform formulation specifically chosen to remain stable over the arena’s repetitive grid texture, with an embedded deep-learning feature front-end and a learned map-quality classifier (Sec. VII).
- A marker-localization subsystem combining planar PnP through a 45° mirror rig with robust geometric-median aggregation and a calibrated bias model (Sec. VIII).
- A from-scratch EKF with a two-stage adaptive gate that rejects the impulsive velocity bursts produced by visual-SLAM loop closures (Sec. IX).
- A priority-weighted map-fusion rule that merges the aerial prior with the AMR’s live map without any registration step (Sec. XII).
- An independent, high-rate automatic-emergency-braking layer (Sec. XIV) and an application-layer security middleware protecting all ROS 2 traffic (Sec. XV).

E. Document Organization

Section II provides theoretical background. Section III states system requirements, and Section IV presents the hardware and software architecture. Section V describes the cooperative mission pipeline that frames the subsystem deep dives in Sections VI–XV. Section XVI reports experimental results, and Section XVII concludes.

II. THEORETICAL BACKGROUND

This section collects the theory the subsystems rely on. Each primer is kept brief; full derivations and the design choices specific to our implementation appear in the corresponding deep-dive section.

A. Differential-Drive Kinematics and Feedback Linearization

A differential-drive (unicycle) robot has configuration (x, y, θ) evolving as $\dot{x} = v \cos \theta$, $\dot{y} = v \sin \theta$, $\dot{\theta} = \omega$, where v, ω are body-frame forward and angular velocity. The robot is non-holonomic: it cannot translate instantaneously along its lateral axis. This makes direct proportional control of the geometric centre ill-posed, because the mapping from (v, ω) to centre velocity is singular. Feedback linearization sidesteps this by controlling a *virtual point* offset a fixed distance R ahead of the centre; the map from (v, ω) to that point’s velocity is invertible for all θ , yielding linear, decoupled error dynamics (Sec. X).

B. The Extended Kalman Filter

The EKF estimates the state of a nonlinear system by alternating a *prediction* step, which propagates the state mean through the nonlinear process model f and the covariance through its Jacobian $F = \partial f / \partial \mathbf{X}$, with *update* steps that correct the prediction using measurements through an observation model—linear in our case, so each update reduces to the ordinary Kalman correction. Each update computes a Kalman gain trading off process and measurement uncertainty. Numerically robust implementations use the *Joseph form* of the covariance update, which preserves symmetry and positive semi-definiteness regardless of gain magnitude (Sec. IX).

C. Bayesian Occupancy Grids

An occupancy grid discretizes the world into cells, each holding the probability that it is occupied. Maintaining this in *log-odds* form, $l = \log \frac{p}{1-p}$, turns Bayesian measurement integration into simple addition: each sensor reading adds an occupied or free increment to the cells it observes, with the cells along a range beam updated by ray casting (Bresenham). Asymmetric increments encode that a single occupied return is more informative than a single free return (Sec. XI).

D. Feature-Based Image Stitching

Stitching aligns overlapping images by detecting keypoints (e.g. SIFT), matching their descriptors, and estimating a geometric transform between frames. Spurious matches are filtered by Lowe’s ratio test and by RANSAC, which fits the transform to the largest consistent inlier set. The transform family matters: a full projective homography has 8 degrees of freedom, whereas a *similarity* transform (translation, rotation, uniform scale) has only 4. For a nadir camera over a planar scene the true motion is a similarity, and constraining the estimate to that family prevents the extra projective degrees of freedom from fitting noise—critical over repetitive texture (Sec. VII).

E. Marker-Based Pose Estimation

ArUco markers are square fiducials whose four corners, detected in the image, provide 2D–3D correspondences for a Perspective- n -Point (PnP) pose solve. For a planar target the problem reduces to a homography from which the pose is recovered in closed form; the Infinitesimal Plane-based Pose Estimation (IPPE_SQUARE) method returns the two candidate poses of the planar ambiguity, disambiguated by reprojection error (Sec. VIII).

F. Graph Search and Trajectory Generation

A* searches a grid graph for a minimum-cost path using an admissible heuristic; on an 8-connected grid the octile distance is the tight admissible heuristic. The resulting waypoint sequence is converted to a smooth, time-parameterized trajectory by fitting an arc-length-parameterized cubic spline and applying a trapezoidal speed profile, yielding continuous reference position and velocity for the tracking controller (Sec. XIII).

TABLE I
FUNCTIONAL REQUIREMENTS

ID	Requirement
FR-1	The AMR shall navigate from start to a marked goal in a GPS-denied arena without human intervention.
FR-2	The UAV shall autonomously survey the arena and build an occupancy map prior to AMR navigation.
FR-3	The system shall localize ArUco goal markers in the world frame with accuracy sufficient for planning.
FR-4	The AMR shall detect and avoid static obstacles in real time.
FR-5	The mission shall execute without a pre-built map; the map is built at runtime.
FR-6	The system shall tolerate communication latency and degrade safely on loss.
FR-7	Mission-critical ROS 2 topics shall be authenticated, and optionally encrypted.
FR-8	The UAV shall fly an autonomous coverage (boustrophedon) survey trajectory under closed-loop position control.
FR-9	The system shall localize the AMR in the world frame from a drone-observed top fiducial, to align the AMR <code>odom</code> frame with <code>world</code> .
FR-10	The AMR shall fuse its proprioceptive and onboard sensors (wheel, IMU, visual odometry) into a single continuous pose estimate.
FR-11	The system shall assess aerial-map quality automatically and fall back to autonomous frontier exploration when the map is rejected.
FR-12	The system shall halt the AMR on imminent collision independently of, and faster than, the planning loop.
FR-13	The AMR shall accommodate slowly moving obstacles through continuous occupancy-map updates (degraded, non-predictive handling).

G. Applied Cryptography Primitives

Message authentication and confidentiality rest on three primitives. A public-key infrastructure (PKI) binds identities to public keys through certificates signed by a trusted certificate authority (CA). RSA-PSS produces asymmetric signatures giving integrity and sender authenticity. AES-256-GCM is a symmetric authenticated cipher giving confidentiality. Combining them—sign then encrypt at the sender, verify before decrypt at the receiver—yields a channel that is both authenticated and confidential (Sec. XV).

III. SYSTEM REQUIREMENTS

A. Functional Requirements

B. Non-Functional Requirements

C. Mechanical and Electronic Requirements

IV. SYSTEM ARCHITECTURE

A. Hardware Platform

The ground robot is built on a Yahboom RDK X3 differential-capable base (quad-core ARM Cortex-A53) carrying an NVIDIA Jetson Orin Nano (8 GB RAM, GPU) as a compute co-processor. Its sensors are an OraDAR MS200 2D LiDAR (360°, 15 Hz, 0.05 m to 20 m), an integrated IMU and wheel encoders read over the SunriseRobot SDK, and a forward facing Intel Realsense D435i camera used for fiducial detection and Visual Simultaneous Localization and Mapping (VSLAM). The aerial agent is a DJI Tello quadrotor with downward and forward cameras; a 45° mirror redirects its

TABLE II
NON-FUNCTIONAL REQUIREMENTS

ID	Requirement
NFR-1	The state estimator shall be implemented from scratch (no off-the-shelf localization package).
NFR-2	The LiDAR occupancy map shall update at ≥ 0.5 Hz.
NFR-3	The planner shall replan within 500 ms of a new map.
NFR-4	The AEB layer shall react to obstacles within ~ 333 ms at the deployed LiDAR rate.
NFR-5	The stitching pipeline shall fit within the Jetson Orin Nano's 8 GB memory.
NFR-6	The system shall run on ROS 2 Humble.
NFR-7	The EKF and tracking-control loops shall run at 100 Hz.
NFR-8	The compute shall be decentralized across at least two independent on-board processors (RDK X3, Jetson Orin Nano).
NFR-9	Inter-machine map and goal topics shall use latched (<code>TRANSIENT_LOCAL</code>) QoS so late-joining nodes receive them.
NFR-10	The security layer shall be globally disengageable through a single kill switch without code changes.
NFR-11	Per-message cryptographic overhead shall remain within the real-time messaging budget.

TABLE III
MECHANICAL AND ELECTRONIC REQUIREMENTS

ID	Requirement
ER-1	The AMR LiDAR scan plane shall sit at 13 cm—above floor discontinuities, below obstacle height.
ER-2	The UAV shall view the arena at nadir via a 45° mirror redirecting its forward camera.
ER-3	The static transform <code>base_footprint</code> $\rightarrow$ <code>lidar</code> shall be $x = 0.07$ m, $z = 0.13$ m.
ER-4	The camera-to-drone transform shall be calibrated once per deployment (roll = π , yaw = 0.0456 rad).

forward camera to a nadir view. Absolute positioning of the UAV is provided by an OptiTrack motion-capture system (sub-millimetre, streamed over VRPN), standing in for the GNSS a field drone would use. Table IV summarizes the platform. **[FIGURE: Labeled photo/diagram of AMR and UAV hardware]**

Compute is split deliberately across the two on-board processors (NFR-8): the RDK X3 runs low-level motor control and the EKF, while the Jetson Orin Nano runs the perception- and planning-heavy workloads (stitching, fusion, planning, orchestration). The LiDAR is mounted so its scan plane lies above floor irregularities yet intersects the 30 cm-tall obstacle boxes (ER-1, ER-3). **[TODO: Insert image of car with all sensors]** Additionally, and providing a fiducial for locating the AMR from the drone view, we equip the vehicle with a 3D printed top mounted base with an Aruco marker.

B. Software Decomposition

The software spans three main repositories: the Jetson stack (`collab_nav_ground-jetson`), the RDK X3 stack

TABLE IV
HARDWARE PLATFORM SUMMARY

Component	Specification
AMR base	Yahboom RDK X3, ROS 2 Humble
AMR co-processor	Jetson Orin Nano, 8 GB, GPU
LiDAR	OraDAR MS200, 360°, 15 Hz
IMU / encoders	Integrated, SunriseRobot SDK
AMR camera	Intel Realsense D435i, ArUco detection and VSLAM
UAV	DJI Tello, 720p, 45° mirror rig
UAV positioning	OptiTrack (VRPN over Ethernet)

TABLE V
SUBSYSTEM-TO-PACKAGE MAPPING

Subsystem	Packages	Host
UAV flight	tello_driver,	Jetson
Aerial mapping	tello_pos_control	Jetson
AMR localization	arena_map_builder,	
	arena_marker_localizer	
	ekf_amr,	RDK X3
	aruco_localizer,	
	amr_optitrack	
Navigation	fusion, world_mapper,	Jetson
	oradar_ros,	
	alignment_node,	
	trajectory_planner,	
	frontier_explorer	
Safety & orch.	emergency_stop,	Jetson
	mission_orchestrator	
Security	ros2_security	All

(collab_nav_ground-rasp), and the security middleware (security_middleware). Functionality is decomposed into the subsystems of Table V, each realized as one or more ROS 2 packages and detailed in Sections VI–XV. **[FIGURE: Subsystem block diagram]**

C. Communication Topology

Two planes carry traffic. The *control plane* uses SSH (Paramiko) for the Jetson to manage the RDK X3’s lifecycle (starting and stopping the AMR service). The *data plane* is ROS 2 DDS over Wi-Fi, sharing all sensor and navigation topics across machines within a common ROS_DOMAIN_ID. The UAV is bridged over its proprietary Tello UDP SDK, and OptiTrack poses arrive over VRPN on a secondary Wi-Fi network. Latched (TRANSIENT_LOCAL) QoS is used for the map and goal topics so that late-joining nodes still receive them (NFR-9). **[FIGURE: Network topology diagram]**

D. Development Methodology

Development followed four incremental stages: (1) basic AMR kinematics, odometry, and LiDAR obstacle detection; (2a) UAV autonomous exploration and OptiTrack positioning, in parallel with (2b) the AMR EKF, mapping, and trajectory control; (3) integration into the cooperative localization-and-handoff pipeline; and (4) full autonomous missions with robustness testing against latency, and communication loss. Although some of them developed in parallel, each stage produced a runnable increment that later stages built on.

V. COOPERATIVE MISSION PIPELINE

The end-to-end mission is sequenced by `mission_orchestrator`, a deterministic twelve-stage finite state machine $M = (Q, \Sigma, \delta, q_0, F)$ with $\Sigma = \{\text{success}, \text{abort}\}$: each stage advances only on success, and any sub-stage failure transitions to a single ABORT sink (Fig. 1). The stages are:

- OptiTrack bring-up.** Check `/optitrack/rigid_body` presence and header, launching the VRPN client if absent (01.a), then sanity-check its contents (frame id and freshness) (01.b).
- Arena map-builder bring-up.** Launch the server and configure the background-image path (02.a) and the online/offline mode (default online) (02.b). Asynchronously, the server will later return an `OccupancyGrid` and marker poses, joined at 04.b.
- Drone routine.** Connect to the Tello Wi-Fi (03.a), launch `tello_driver` (03.b), run pre-flight checks (live camera, battery above minimum) (03.c), launch the take-off and scanning routine (03.d), and—if online—stream processed frames to the map builder (03.e). The orchestrator then observes the flight state transitions with per-stage timeouts (03.f), waits for the recorded `scan.mp4` and telemetry (03.g), and verifies video integrity with `ffmpeg` (03.h).
- Aerial marker localization (concurrent with map build).** A `BuildArenaMap` goal is dispatched in the *background* while the `arena_marker_localizer` service is launched (04). The orchestrator calls `/localize_markers` as the *orientation* source, running while the map builds (04.a), then joins the map-builder result as the *position* source and publishes `/aruco/amr/pose` (position from the map builder, orientation from the localizer; an all-NaN pose if unavailable, which the alignment node reads as “no data” and falls back to the identity transform) (04.b). Finally the map-quality classifier runs on the `BuildArenaMap` diagnostic features (04.c): on PASS it publishes the `OccupancyGrid` to `/drone/map` and the goal to `/aruco/goal/pose`; on FAIL it publishes a border-template grid (no goal pose), and the frontier-exploration fallback is armed later (11.b) and triggered at 12.a.
- Visual SLAM.** Verify the RealSense D435i (05.a), start the container and launch Isaac ROS cuSLAM (05.b), and confirm valid tracking odometry (05.c).
- AMR bring-up.** Ping (06.a) and SSH into the RDK X3 (06.b), start the `amr_bringup` service and verify it is active (06.c), and wait for the IMU stream (06.d).
- Emergency stop.** Launch `emergency_stop` and verify `/amr/emergency_stop` is inactive (07.a).
- AMR marker localization.** Launch `aruco_localizer` feeding the EKF; the camera source follows the VSLAM toggle (RealSense colour when disabled, the VSLAM IR stream when

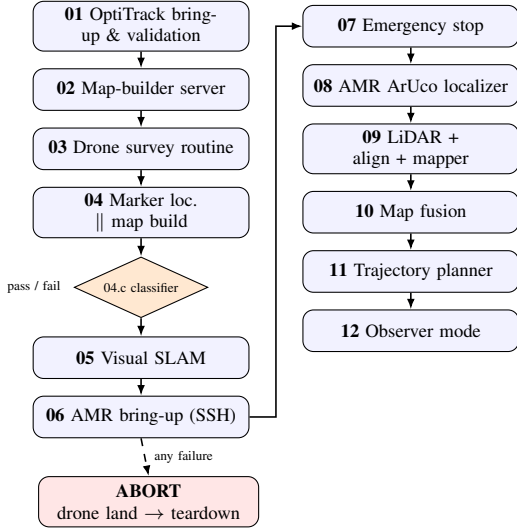


Fig. 1. Mission orchestrator state machine. Stages advance left-to-right, top-to-bottom (01–06 then 07–12). The map-quality classifier (04.c) is the sole branch: PASS routes the planner to `/drone/map` with a goal pose; FAIL publishes a border-template map and arms the frontier-exploration fallback (11.b, triggered at 12.a). Any sub-stage failure transitions to the ABORT sink, which lands the drone before tearing down.

enabled) (08.a).

09. **Mapping.** Launch the OraDAR LiDAR (09.a), the `alignment_node` mapping `/aruco/amr/pose` to the `world→odom` transform (09.b), and the odometry-based occupancy mapper (09.c).
10. **Map fusion.** Launch fusion overlaying the drone and AMR-built maps; fire-and-forget, as fusion only emits once `/drone/map` exists.
11. **Trajectory planner.** Launch the A* planner and spline follower, with `map_topic = /drone/map` on PASS or `/fusion/map` on FAIL. On FAIL only, the frontier-exploration stack is launched and idles until triggered (11.b).
12. **Observer mode.** Enter telemetry logging; if in the frontier-exploration (map FAIL) mode, send the start signal on `/map_fail_fallback/start` once bring-up completes (12.a).

Stage 04 exploits structured concurrency: map building and marker localization share inputs but are computationally independent, so pipelining them reduces wall-clock latency from $T_{\text{stitch}} + T_{\text{PNP}}$ to $\max(T_{\text{stitch}}, T_{\text{PNP}})$. The classifier at 04.c is the single decision point that branches the otherwise-linear pipeline: a passing map drives goal-directed planning, while a failing map degrades gracefully to frontier exploration over the fused map rather than aborting. The abort sequence prioritizes drone safety (land command first), then tears down nodes gracefully (SIGINT→SIGKILL) and closes SSH sessions.

[FIGURE: Pipeline data-flow with message types at stage boundaries]

VI. UAV FLIGHT CONTROL

A. Role and Requirements

`tello_pos_control` flies the survey pattern that Section VII stitches into a map, satisfying FR-2. It must track a world-frame scan trajectory using OptiTrack pose feedback, despite the Tello exposing only a *body-frame* velocity interface for its control, and must abort safely on loss of tracking (FR-6) or direct orchestrator’s command.

B. Theory and Design Rationale ✓

1) *Flight state machine:* Flight proceeds through six states: pre-flight, takeoff/home, stabilize at home, trajectory, return home, and normal land, with a seventh fault-land state reachable from the stabilize, trajectory, and return states when the tracking fault detector fires at detection of unphysical pose data.

2) *Position controller:* The world-frame position error is $\mathbf{e} = \mathbf{p} - \mathbf{p}^*$. Each axis uses PID with feedforward and anti-windup on the integral term,

$$I_{x,k} = \text{clip}(I_{x,k-1} + e_x \Delta t, -I_x^{\max}, I_x^{\max}), \quad (1)$$

$$v_x^W = -k_{p,x} e_x - k_{d,x} \tilde{v}_x - k_{i,x} I_x + v_{ff,x}, \quad (2)$$

where $\tilde{v}_x$ is an EMA-smoothed finite-difference velocity. Because the Tello accepts body-frame commands, the world-frame command is rotated through the measured yaw ψ :

$$\begin{pmatrix} v_x^B \\ v_y^B \end{pmatrix} = \mathbf{R}(\psi)^\top \begin{pmatrix} v_x^W \\ v_y^W \end{pmatrix} = \begin{pmatrix} \cos \psi & \sin \psi \\ -\sin \psi & \cos \psi \end{pmatrix} \begin{pmatrix} v_x^W \\ v_y^W \end{pmatrix}. \quad (3)$$

The command is then saturated in a direction-preserving manner at $v_{\max}$, and yaw is held with $\omega_z = \text{clip}(-k_{p,\psi} \text{atan2}(\sin(\psi - \psi_d), \cos(\psi - \psi_d)), -1, 1)$.

3) *OptiTrack fault detector:* Tracking can fail in an *alternating-ghost* mode, interleaving valid and corrupted samples, which defeats a naive consecutive-bad-sample counter. We instead monitor a sliding-window bad-sample ratio. With $b_k = \mathbf{1}[\|\mathbf{p}_k - \mathbf{p}_{k-1}\| > \delta_{\text{thr}}]$, a fault is declared (and latched) when

$$\hat{\rho}_k = \frac{1}{W} \sum_j b_j > \rho_{\text{thr}}. \quad (4)$$

The detector is active only in the stabilize, trajectory, and return states.

4) *Scan trajectory:* Survey waypoints form a boustrophedon (lawnmower) raster. The path is arc-length parameterized, $t_i = s_i/v$ with $s_i = \sum d_j$ the cumulative chord length, and interpolated by a clamped cubic spline with zero endpoint velocities. The analytic spline derivative supplies the feedforward velocity.

C. Implementation

The node subscribes to OptiTrack pose and publishes body-frame velocity commands to `tello_driver`, advancing the state machine on a fixed timer. **[TODO: Topic/parameter table.]**

D. Traceability and Tests

Satisfies FR-2, FR-6, FR-8. **[TODO: Flight tests: trajectory tracking error under nominal tracking; fault-injection test forcing the alternating-ghost mode.] [FIGURE: Commanded vs. OptiTrack trajectory]**

VII. AERIAL MAPPING AND IMAGE STITCHING

A. Role and Requirements

`arena_map_builder` converts downward UAV video into a metric occupancy grid at 5cm/cell, plus marker positions and a diagnostic feature vector. It satisfies FR-2 and FR-5 and must fit the Jetson memory budget (NFR-5).

B. Theory and Design Rationale

1) *Similarity transform over full homography*: Top-down flight over a planar arena is a 4-DOF rigid-plus-scale motion. A full 8-DOF homography has extra degrees of freedom that RANSAC will use to fit the repetitive blue-tape grid, producing compounding “fan” artifacts across the hundreds of frames of a survey. We therefore estimate a similarity transform

$$\mathbf{H}_{\text{sim}} = \begin{pmatrix} a & -b & t_x \\ b & a & t_y \\ 0 & 0 & 1 \end{pmatrix}, \quad a = s \cos \theta, \quad b = s \sin \theta, \quad (5)$$

via `estimateAffinePartial2D` with RANSAC, promoted to 3×3 for composition onto the global canvas.

2) *Feature extraction with exclusion masking*: The production front-end is SIFT (5000 keypoints/frame, 128-D descriptors). The arena’s blue adhesive grid is masked out by an HSV threshold (dilated 5px) before detection: without this mask, RANSAC latches onto visually identical inter-cell tape junctions and converges on the wrong cell. A GPU alternative front-end uses SuperPoint (256-D) with LightGlue matching, exported to ONNX and run on the Jetson GPU—the embedded deep-learning component of the system.

3) *Matching pipeline*: Frame pairs are matched by (i) k -nearest-neighbour ($k=2$) L2 matching, (ii) Lowe’s ratio test at $r = 0.65$, (iii) a mutual cross-check requiring the ratio test to pass in both directions, and (iv) a median-absolute-deviation (MAD) displacement pre-filter that removes wrong-cell clusters *before* RANSAC:

$$\boldsymbol{\mu} = \text{median}_i(\boldsymbol{\Delta}_i), \quad \text{MAD} = \text{median}_i \|\boldsymbol{\Delta}_i - \boldsymbol{\mu}\|, \quad (6)$$

keeping match i iff $\|\boldsymbol{\Delta}_i - \boldsymbol{\mu}\| < \kappa \cdot \text{MAD}$ with $\kappa = 4.0$. Survivors feed a similarity RANSAC (threshold 3px, 5000 iterations, confidence 0.999).

4) *Canvas and blending*: A fixed 8000×8000 canvas is pre-allocated; each frame is warped only over its region of interest (~ 6 MB vs. ~ 140 MB for the full canvas). Frames are merged with a 4-level Laplacian pyramid blend whose weight $\alpha(p) = d_{\text{new}} / (d_{\text{new}} + d_{\text{old}})$ comes from a dual distance transform, placing the seam along the iso-depth contour.

5) *Obstacle transfer and consistency scoring*: Obstacles (pink) and walls (green/brown) are transferred by two independent projections (bounding-box and grid), whose centroid and area agreement form confidence proxies s_A, s_B ; a perturbation-stability proxy s_C completes a weighted score

$$c = \frac{w_A s_A + w_B s_B + w_C s_C}{w_A + w_B + w_C}, \quad (7)$$

with $(w_A, w_B, w_C) = (0.45, 0.25, 0.30)$. Occupancy is rasterized with confidence-weighted halos of width $t(c) = t_{\text{base}} e^{-3c}$ interpolating from free to occupied around each obstacle core.

6) *Learned map-quality gate*: A RandomForest classifier consumes a 45-feature diagnostic vector and outputs a binary “map acceptable” decision $P(\text{pass} \mid \mathbf{f}) \geq \tau$. Inference is pure NumPy (a serialized `forest.npz`, no scikit-learn at runtime); missing features map to a -1 sentinel for safe-fail. A rejected map triggers the frontier-exploration fallback of the mission FSM.

C. Implementation

The builder runs online (incrementally during flight, hiding stitching latency behind flight time) or offline (post-landing on saved video). Memory is bounded (NFR-5) by a streaming frame generator, the pre-allocated canvas, ROI warping, a keyframe cap of 20, and periodic `malloc_trim`; peak working set is ~ 287 MB. A 335-configuration parameter sweep fixed the deployed baseline (SIFT, $r=0.65$, $\kappa=4.0$, lookback 8, keyframe interval 10, processing scale 0.5, blue-tape exclusion).

D. Traceability and Tests

Satisfies FR-2, FR-5, FR-11, NFR-5. **[TODO: Coverage and obstacle TP/FP metrics; ablations: mask on/off, SIFT vs. SuperPoint.] [FIGURE: Stitched map with/without exclusion mask] [FIGURE: Occupancy grid with confidence halos]**

VIII. AERIAL MARKER LOCALIZATION

A. Role and Requirements

`arena_marker_localizer` estimates the world-frame pose of each ArUco goal marker from the UAV’s aerial video and synchronized OptiTrack log, satisfying FR-3. Position is taken from the stitcher (whose metric pixel projection is more accurate) and *orientation* from PnP, which depends only on relative image geometry and is unaffected by map-origin shifts.

B. Theory and Design Rationale

1) *Mirror rig and camera transform*: A 45° mirror redirects the forward camera to nadir. The calibrated camera-to-drone transform applies a roll of π (mapping the forward optical axis to downward) and a yaw of 0.0456 rad (2.6° mounting misalignment), with a small translational lever arm.

2) *Planar PnP*: Each detected marker yields four corner correspondences solved by IPPE_SQUARE, a closed-form planar method that reduces the 3D–2D problem to a homography and recovers two candidate poses (the planar ambiguity), selecting the one with lower reprojection error. Detections with reprojection error above 4px are discarded.

3) *Transform chain*: A marker pose is composed along a four-link chain,

$$\mathbf{T}_{\text{map} \leftarrow \text{marker}} = \mathbf{T}_{\text{map} \leftarrow \text{opti}} \mathbf{T}_{\text{opti} \leftarrow \text{drone}}(t) \mathbf{T}_{\text{drone} \leftarrow \text{cam}} \mathbf{T}_{\text{cam} \leftarrow \text{marker}}(t), \quad (8)$$

with all links as 4×4 homogeneous transforms and the OptiTrack log aligned row-by-row to video frames.

4) *Robust aggregation*: Each marker accrues 120–180 observations. These pass a per-axis and circular MAD gate ($k = 3.5$), after which position is aggregated by the Weiszfeld geometric median (breakdown point $\sim 50\%$), $\mathbf{x}^{(k+1)} = \sum_i w_i \mathbf{p}_i / \sum_i w_i$ with $w_i = 1/\|\mathbf{p}_i - \mathbf{x}^{(k)}\|$, and yaw by the circular median $\hat{\theta} = \text{atan2}(\text{median} \sin \theta_i, \text{median} \cos \theta_i)$.

5) *Bias calibration*: A four-term linear error model—map-origin shift, camera lever arm, radial scale bias, and yaw misalignment—is fit by iteratively reweighted least squares with Huber weights and physical bounds, validated by leave-one-scan-out cross-validation. An SVD condition-number check detects attitude degeneracy and falls back to a reduced model.

C. Implementation

The subsystem exposes a `/localize_markers` service invoked in mission Stage 04 as the orientation source, joined with the map-builder position to publish `/aruco/amr/pose` and (on classifier pass) `/aruco/goal/pose`. **[TODO: Service/message interface details.]**

D. Traceability and Tests

Satisfies FR-3, FR-9. **[TODO: Position and yaw error after calibration; cross-validation residuals.]** **[FIGURE: Transform-chain and mirror-rig geometry diagram]**

IX. AMR STATE ESTIMATION: EXTENDED KALMAN FILTER

A. Role and Requirements

`ekf_amr` is the AMR’s state estimator, fusing wheel-encoder velocity, IMU gyroscope, and visual-odometry (VIO) forward velocity into a continuous pose on `/amr/ekf/odom` and the `odom` $\rightarrow$ `base_footprint` transform. It is implemented from scratch (NFR-1). The GPS-denied design (no absolute-positioning markers on the AMR) forces the estimate to be built from proprioceptive and onboard sensing alone; the noisy, loop-closure-spiked VIO stream demands adaptive gating that stock packages do not provide.

B. Theory and Design Rationale

1) *State and process model*: The state $\mathbf{X} = [x, y, \theta, v, \omega]^\top$ carries velocity as explicit states so heterogeneous velocity sources—and future absolute-position fixes—all enter through standard Kalman updates. Between steps it evolves under a unicycle model with a constant-velocity assumption,

$$f(\mathbf{X}) = [x + T_s v \cos \theta, y + T_s v \sin \theta, \theta + T_s \omega, v, \omega]^\top, \quad (9)$$

Algorithm 1 P1 adaptive EMA gate (per VIO sample)

```

in_burst  $\leftarrow \text{mean}(\text{recent\_window}) \geq 0.30$ 
 $\sigma^2 \leftarrow \max(0, \text{ema\_sq} - \text{ema}^2)$ 
 $\tau \leftarrow \max(2.0\sqrt{\sigma^2}, 0.17)$ 
spike  $\leftarrow |v_x - \text{ema}| > \tau$ 
if in_burst or spike then
    v_out  $\leftarrow \text{last\_good}$ 
    update ema, ema_sq with  $\alpha_{\text{burst}}$ , clip( $v_x$ ,  $-0.5$ ,  $0.5$ )
else
    last_good  $\leftarrow v_x$ ; v_out  $\leftarrow v_x$ 
    update ema, ema_sq with  $\alpha$ ,  $v_x$ 
end if
append spike to recent_window; return v_out

```

with Jacobian

$$F = \begin{pmatrix} 1 & 0 & -T_s v \sin \theta & T_s \cos \theta & 0 \\ 0 & 1 & T_s v \cos \theta & T_s \sin \theta & 0 \\ 0 & 0 & 1 & 0 & T_s \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}, \quad (10)$$

and process noise $Q = \text{diag}(10^{-4}, 10^{-4}, 10^{-4}, 0.05, 0.05)$: pose is trusted to propagate, while loose velocity noise lets measurements correct (v, ω) freely. The prediction is $\hat{\mathbf{X}}^- = f(\mathbf{X}_{k-1})$, $P^- = F P F^\top + Q$, with heading wrapped to $(-\pi, \pi]$.

2) *Measurement updates*: Three updates fire sequentially each step (wheel, IMU, VIO), skipping sensors with no new reading. Wheel odometry observes (v, ω) with $R_{\text{wheel}} = \text{diag}(0.2, 0.2)$ (slip and quantization). The IMU observes ω after subtracting a calibrated bias $b = 0.002618 \text{ rad s}^{-1}$, with a tight $R_{\text{imu}} = \text{diag}(0.001)$. VIO contributes only its forward velocity $v_{x,\text{vio}}$ ($R_{\text{vio}} = \text{diag}(0.1)$); its position is discarded because VSLAM position drifts and jumps at loop closures.

3) *Adaptive VIO gate*: A fixed-cutoff Butterworth low-pass filter could not suppress loop-closure spikes—its transient response to a sharp spike propagated into the filter and caused sustained position deviations. The deployed `ema_gate` is a two-stage adaptive filter. Stage P0 holds the last good value when $|v_x| > 0.5 \text{ ms}^{-1}$ (the robot’s mechanical limit). Stage P1 tracks online EMA estimates of mean and variance,

$$\hat{\mu}_k = \alpha v_k + (1 - \alpha) \hat{\mu}_{k-1}, \quad \hat{v}_k^2 = \alpha v_k^2 + (1 - \alpha) \hat{v}_{k-1}^2, \quad (11)$$

$\hat{\sigma}^2 = \max(0, \hat{v}_k^2 - \hat{\mu}_k^2)$, and gates against $\tau_k = \max(2.0 \hat{\sigma}_k, 0.17)$, the floor preventing collapse near stand-still. Because spikes arrive in clusters, a 10-sample window of spike decisions triggers *burst mode* above a 0.30 spike fraction, during which all samples are rejected and the statistics update slowly ($\alpha_{\text{burst}} = 0.02$). A 20-sample warmup seeds the statistics.

4) *Covariance update and anchoring*: All updates use the Joseph form $P^+ = (I - KH)P^-(I - KH)^\top + KRK^\top$, preserving symmetry and positive semi-definiteness to machine precision. No active sensor observes absolute (x, y) , so the filter dead-reckons between fixes; the 5-state form admits

TABLE VI
EKF DEPLOYED PARAMETERS (SELECTED)

Parameter	Value	Meaning
sampling_time	0.01 s	step period (100 Hz)
Q diag	$[10^{-4}, 10^{-4}, 10^{-4}, 0.05, 0.05]$	process noise
R_{wheel}	$[0.2, 0.2]$	wheel $[v, \omega]$
R_{imu}	$[0.001]$	gyro $[\omega]$
R_{vio}	$[0.1]$	VIO $[v_x]$
p0_vx_max	0.5 m s^{-1}	physical limit
pl_n_sigma	2.0	gate width
pl_burst_thresh	0.30	burst entry fraction

an ArUco position update $H_{\text{aruco}} = [I_{3 \times 3} | 0_{3 \times 2}]$ with no structural change.

C. Implementation

The EKF step runs on a fixed 100 Hz timer independent of sensor arrival; callbacks buffer readings behind `_new` flags, and a step with no new data returns early to avoid spurious drift. Parameters appear in Table VI.

D. Traceability and Tests

Satisfies FR-1, FR-10, NFR-1, NFR-7. **[TODO: Position RMSE vs. OptiTrack ground truth; VIO spike-rejection ablation (gate on/off).]** **[FIGURE: EKF state and covariance traces over a run]** **[FIGURE: Raw vs. gated VIO velocity]**

X. AMR DRIVE STACK: ODOMETRY AND CONTROL

A. Role and Requirements

`amr_bringup` is the RDK X3 hardware layer: `driver_node` (hardware abstraction), `wheel_odometry_node` (dead-reckoning), and `controller_node` (reference tracking). Together they close a sensing–estimation–control loop at the drive level, decoupled from planning.

B. Theory and Design Rationale

1) *Hardware abstraction*: `driver_node` wraps the SunriseRobot SDK and is the only node touching hardware. It forwards `/amr/cmd_vel` to `set_car_motion`, reads wheel velocity, IMU, and battery at 100 Hz, and enforces emergency stop: a `False`→`True` transition immediately commands zero motion and a three-beep alert; the clear transition resumes commands. The IMU is mounted non-canonically, so a fixed axis permutation and sign flip maps the SDK frame to the ROS body frame,

$$(a_x, a_y, a_z)_{\text{ROS}} = (-a_x^s, a_z^s, a_y^s), \quad (12)$$

and identically for the gyroscope; the magnetometer is unused.

2) *Dead-reckoning odometry*: Though mechanically a mecanum platform, the base is treated as a differential-drive robot: lateral velocity is discarded ($\dot{y}_{\text{body}} = 0$ by construction), since planning never commands lateral motion. The pose integrates by Euler,

$$x_{k+1} = x_k + v_x \cos \theta_k \Delta t, \quad y_{k+1} = y_k + v_x \sin \theta_k \Delta t, \quad (13)$$

$\theta_{k+1} = \text{wrap}(\theta_k + \omega_z \Delta t)$, with a Δt guard ($0 < \Delta t \leq 1$ s) and unit calibration scales (the SDK velocities need no correction at the arena scale).

3) *Feedback-linearizing controller*: The planner publishes a world-frame reference (x_r, y_r, θ_r) with velocity $(\dot{x}_r, \dot{y}_r)$. Controlling the centre directly is singular for the unicycle, so the controller acts on a virtual point $R = 0.2$ m ahead, $x_R = x + R \cos \theta$, $y_R = y + R \sin \theta$. Virtual proportional-plus-feedforward inputs $u_1 = k_{px}(x_r - x_R) + \dot{x}_r$, $u_2 = k_{py}(y_r - y_R) + \dot{y}_r$ ($k_{px} = 1.0$, $k_{py} = 2.0$) are mapped to body commands by inverting

$$\begin{pmatrix} \dot{x}_R \\ \dot{y}_R \end{pmatrix} = \underbrace{\begin{pmatrix} \cos \theta & -R \sin \theta \\ \sin \theta & R \cos \theta \end{pmatrix}}_{T(\theta)} \begin{pmatrix} v \\ \omega \end{pmatrix}, \quad (14)$$

which is invertible everywhere ($\det T = R \neq 0$), giving $v = u_1 \cos \theta + u_2 \sin \theta$ and $\omega = (-u_1 \sin \theta + u_2 \cos \theta)/R$ and the decoupled error dynamics $\dot{e}_x = -k_{px}e_x$, $\dot{e}_y = -k_{py}e_y$. Drivetrain stiction is compensated by a deadband offset, $v_{\text{cmd}} = \text{sgn}(v)(|v| + 0.15)$ for $|v| > 0.001 \text{ m s}^{-1}$.

C. Implementation

The controller reads pose from the world $\rightarrow$ `base_footprint` transform rather than a fixed odometry topic, making it localization-agnostic; all three nodes run 100 Hz loops.

D. Traceability and Tests

Satisfies FR-1, FR-4 (emergency-stop actuation), NFR-7. **[TODO: Step-response and tracking-error tests.]**

XI. LiDAR SENSING AND OCCUPANCY MAPPING

A. Role and Requirements

`oradar_ros` drives the LiDAR and `world_mapper` maintains the AMR’s running occupancy map, jointly satisfying FR-4 and NFR-2.

B. Theory and Design Rationale

1) *LiDAR driver*: The OraDAR MS200 streams `OradarLidarFrame` packets (12 points each, CRC-8 checked, distances in mm, angles in centi-degrees). The driver converts clockwise to counter-clockwise ($\text{angle} \rightarrow 360 - \text{angle}$), gates ranges to 0.05 m to 20 m, maps returns onto a regular angular grid (nearest return wins on collision), and fills single-cell gaps by interpolation. A static transform places the sensor at $x = 0.07$ m, $z = 0.13$ m on `base_footprint`; the scan timestamp is the acquisition start, bounding positional uncertainty to ≤ 10 mm at operating speed.

2) *Bayesian occupancy update*: Each cell holds a log-odds value updated additively, $l_t = l_{t-1} + l_{\text{sensor}} - l_0$ with $l_0 = 0$, using $l_{\text{occ}} = +0.85$ and $l_{\text{free}} = -0.40$. The 2.1:1 asymmetry encodes that an occupied return is more informative than a free one. Beams are integrated by Bresenham ray casting: cells along the beam receive the free increment and the terminal cell (if in range) the occupied increment. Values saturate, so cells require several consistent observations to commit (~ 6 hits or 13 misses).

3) *Handling of dynamic obstacles*: The arena contains only static obstacles, so no motion model or prediction is implemented. A slowly moving obstacle is nonetheless handled—sub-optimally but without any dedicated machinery—by the continuous log-odds update itself: as the obstacle vacates a cell, repeated free returns drive that cell’s log-odds back below the occupancy threshold, while its new location accumulates occupied returns. The hit/miss asymmetry and saturation bounds set how quickly the map forgets a departed obstacle; this lag makes the scheme adequate only for motion slow relative to the mapping rate (FR-13).

C. Implementation

The driver runs at the deployed 15 Hz; `world_mapper` integrates scans on a 50 Hz timer along the `world` $\rightarrow$ `odom` $\rightarrow$ `base_footprint` $\rightarrow$ `lidar chain`. The `world` $\rightarrow$ `odom` transform is published by the `alignment_node` from the drone-observed AMR fix on `/aruco/amr/pose` (Stage 09.b); when that pose is unavailable it arrives as all-NaN, which the alignment node reads as “no data” and replaces with the trivial identity transform.

D. Traceability and Tests

Satisfies FR-4, FR-9, FR-13, NFR-2. **[TODO: Mapping fidelity vs. ground-truth obstacle layout.]** **[FIGURE: Live occupancy map during a run]**

XII. MAP FUSION

A. Role and Requirements

`fusion` merges the aerial prior with the AMR’s live LiDAR map into the costmap the planner consumes (FR-4, FR-5).

B. Theory and Design Rationale

Both maps are published in the same `world` frame at identical resolution and origin by construction, so *no registration is needed*. Fusion is a single vectorized elementwise pass with a four-case priority rule (drone threshold 65, default fill 25):

$$G_F = \begin{cases} 100 & G_D \geq 65 \text{ (drone obstacle, immutable)} \\ G_A & G_D = -1 \text{ (drone unknown, defer to AMR)} \\ \max(25, G_A) & G_D < 65, G_D \neq -1, G_A \neq -1 \\ 25 & \text{otherwise.} \end{cases} \quad (15)$$

Drone free space is treated as 25 rather than 0 because aerial stitching is not certain enough to assert free; the AMR map can only *raise* occupancy, never clear a drone obstacle. An earlier

ICP-based registration was abandoned: early in a mission the AMR map is too sparse for stable coarse-search peaks, and exact pre-alignment made registration unnecessary.

C. Implementation

The drone map uses `TRANSIENT_LOCAL` QoS so the node receives it even if it starts late; the fused map is republished reactively on each AMR update.

D. Traceability and Tests

Satisfies FR-4, FR-5, NFR-9. **[TODO: Fused-map correctness vs. individual sources.]** **[FIGURE: Drone / AMR / fused maps side by side]**

XIII. PATH PLANNING AND TRAJECTORY EXECUTION

A. Role and Requirements

`trajectory_planner` plans a collision-free path to the goal and emits a trackable reference for the drive stack (FR-1, FR-4, NFR-3).

B. Theory and Design Rationale

The fused costmap is inflated by an exponential look-up table $c(d) = 99 e^{-k(d-r_{\text{robot}})}$ ($k = 3.5$, saturating to 100 inside the robot radius). A^* searches the 8-connected grid with the octile heuristic

$$h = \max(\Delta x, \Delta y) + (\sqrt{2} - 1) \min(\Delta x, \Delta y), \quad (16)$$

on occupancy-weighted edge costs. Replanning triggers on goal change, path blockage, elapsed time, or a proximity score $\sigma_{\text{prox}} = \sum_i e^{-d_i/r}$ exceeding a threshold. The way-point path is smoothed by an arc-length -parameterized cubic spline and given a trapezoidal speed profile (ramp distance $d_{\text{ramp}} = v^2/2a = 0.225$ m)—the dynamic speed adjustment of the extra-credit criteria. The resulting reference is tracked by the same feedback-linearizing controller as Section X.

C. Implementation

Inputs are the fused `/drone/map` and the ArUco goal pose; the node publishes the time-parameterized reference to the drive controller.

D. Traceability and Tests

Satisfies FR-1, FR-4, FR-13, NFR-3. **[TODO: Path optimality, replanning latency, dynamic-obstacle response.]** **[FIGURE: Planned path over inflated costmap]**

XIV. PHYSICAL SAFETY: AUTOMATIC EMERGENCY BRAKING

A. Role and Requirements

`emergency_stop` is an independent, high-rate safety layer decoupled from planning (FR-4, NFR-4).

B. Theory and Design Rationale

A blind-spot angular mask excludes $[120^\circ, 240^\circ]$ (chassis structure). An N -consecutive streak detector commands STOP after $n = 5$ consecutive scans report an obstacle within 0.3 m, and clears after n clean scans; at the deployed 15 Hz this is a reaction time of $\tau \approx 5/15 \approx 333$ ms. Multiple trigger reasons combine in an OR state machine. Safety holds when the stopping distance bound $d_{\min} > v \cdot (n/f_s) + d_{\text{brake}}$ is satisfied.

C. Implementation

The node raises `/amr/emergency_stop`, which `driver_node` honors by zeroing motion; the mission FSM health-checks the node at Stage 07.

D. Traceability and Tests

Satisfies FR-4, FR-12, NFR-4. **[TODO: Measured response time; false-trigger rate.]**

XV. NETWORK SECURITY MIDDLEWARE

A. Role and Requirements

`ros2_security` authenticates and optionally encrypts ROS 2 traffic without DDS-Security or SROS2, satisfying FR-7. It is opt-in per node, with a global kill switch and mixed-trust support.

B. Theory and Design Rationale

A self-signed CA (RSA-4096) issues per-node RSA-2048 X.509 certificates whose common name equals the node name; all nodes load the CA certificate to verify peers. Three levels are offered: NONE (pass-through), SIGN (RSA-PSS over the CDR payload), and SIGN_ENCRYPT (AES-256-GCM then RSA-PSS over the ciphertext). At publish, the payload is serialized, optionally encrypted with a random 12-byte nonce, signed, and wrapped in a `SecureEnvelope` (sender id, timestamp, signature, payload, nonce). At subscribe, the signature is verified *before* any decryption (no decrypt oracle), the sender is checked against the trusted set, the `min_level` policy is enforced, the timestamp is checked against a replay window, and only then is the payload decrypted and deserialized. The AES key is derived deterministically as SHA-256(public-key DER)—the only key material both parties share—so no secret distribution is required. A `LegacyRelayNode` bridges secured and native topologies, and the `ROS2_SECURITY_DISABLED` kill switch degrades the whole system to native ROS 2 uniformly (a mixed state would cause a DDS type mismatch).

C. Implementation

Security is added by a `SecureNodeMixin` drop-in; policy resolves by priority (kill switch > explicit code argument > YAML policy > global default). Certificates are minted once per deployment.

D. Traceability and Tests

Satisfies FR-7, NFR-10, NFR-11. **[TODO: Unit and integration suite results: tamper, replay, and wrong-sender rejection; SIGN_ENCRYPT round-trip; per-message overhead.]**

XVI. EXPERIMENTAL RESULTS

A. Experimental Setup

[TODO: Arena, obstacle configurations, marker placement, ground-truth source (OptiTrack), number of runs.]

B. Aerial Mapping Accuracy

[TODO: Coverage, obstacle TP/FP, marker localization error after calibration.]

C. State Estimation Accuracy

[TODO: EKF position RMSE vs. OptiTrack; VIO-gate effectiveness.]

D. Navigation Performance

[TODO: Goal-reach success rate, path-tracking error, collision count, mission wall-clock time.]

E. Safety and Security Validation

[TODO: AEB response time; security test-suite results and overhead.]

F. Discussion

[TODO: Quantitative improvement metrics; robustness observations; limitations (dead-reckoning drift between fixes, static gyro bias, sequential-update approximation).]

XVII. CONCLUSION AND FUTURE WORK

We presented a cooperative UAV-AMR system that navigates to a goal in a GPS-denied, initially unmapped arena by having an aerially positioned drone build the map and localize the goal, then handing that prior to a ground robot that fuses it with onboard sensing. The design satisfies the project's functional and non-functional requirements through a from-scratch EKF, a similarity-transform stitching pipeline with embedded deep learning, robust marker localization, registration-free map fusion, an independent emergency-braking layer, and an application-layer security middleware, all integrated in ROS 2 across three networked platforms. **[TODO: Restate headline results.]** Future work includes more frequent ArUco re-anchoring to bound dead-reckoning drift, online gyro-bias estimation, dynamic-obstacle prediction in the planner, and scaling to larger multi-robot teams.

ACKNOWLEDGMENT

[TODO: Advisors; OMRON Automation; Nuclea Solutions; Tecnológico de Monterrey.]

REFERENCES

- [1] [TODO: ROS 2 / DDS reference.]
- [2] [TODO: Thrun, Burgard, Fox — Probabilistic Robotics (EKF, occupancy grids).]
- [3] [TODO: Lowe — Distinctive image features (SIFT).]
- [4] [TODO: Fischler & Bolles — RANSAC.]
- [5] [TODO: Collins & Bartoli — IPPE planar pose.]
- [6] [TODO: DeTone et al. — SuperPoint.]
- [7] [TODO: Lindenberger et al. — LightGlue.]
- [8] [TODO: Hart, Nilsson, Raphael — A*.]
- [9] [TODO: Weiszfeld — geometric median.]
- [10] [TODO: Project repositories (jetson / rasp / security_middleware).]